NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY
Faculty of Computing and Informatics
School of Computing
Department of Software Engineering

13 Jackson Kaujeua Street
Private Bag 13388
Windhoek
NAMIBIA

T: +264 61 207 2052
F: +264 61 207 9052
E: dse@nust.na
W: www.nust.na

PAP521S – PROGRAMMING IN PRACTICE

PROJECT A: DEVELOPING A MUNICIPAL FINANCIAL MANAGEMENT SYSTEM

PROJECT REPORT

DATE: 04 OCTOBER 2026

GROUP MEMBERS		
No.	**Student number**	**Full Name**
1.	226002306	Elizabeth K Nauyoma
2.	226111733	Ewan Lombaard
3.	225000725	Brooklyn September
4.	226082075	Max M.T Kasinga
5.	226017060	Mandlenkosi Sibanda
6.	226101177	Monika Stefanus
7.	225163993	Elifas Nantinda *(Did not complete assigned task and left the group on the submission due date after approximately 95% of the project had been completed.)*

GitHub link: https://github.com/226002306-NauyomaE/PAP521S-Project

1. Introduction

The aim of this project is to develop a municipal financial management system for managing various financial aspects of a municipality. The system is designed to manage employees' salaries, departmental budgets, suppliers, and municipal assets, while also generating reports for each of these areas. The program was developed using a modular approach, with the system divided into separate modules to improve organisation, maintainability, and functionality.

2. Problem description

Municipalities need to manage large amounts of financial and administrative information, including employee salary information, departmental budgets, suppliers, and municipal assets. Managing these areas without a well-structured system can make it difficult to organize information, perform calculations, search for records, monitor budgets, and generate useful reports to support evidence-based decision-making. Therefore, there is a need for an integrated system that can manage these financial activities efficiently within a single, organized application.

3. System objectives

The main objective of the Municipal Financial Management System is to provide an integrated application consisting of separate modules for employee salary management, budget management, supplier management, asset management, and reporting. The system allows relevant information to be entered, managed, searched, processed, and displayed through a central menu-driven application.

The system's specific objectives are:

- To manage employee salary information.
- Managing departmental budgets and expenditure.
- To maintain supplier information.
- To maintain municipal asset information.
- To generate basic reports for different management areas.

4. System features

The Municipal Financial Management System is a menu-driven application consisting of several interconnected modules. The main features of the system are:

- **Main Menu:** Provides navigation to all system modules and the exit option.
- **Employee Management:** Allows users to add and display employee information, search for employees, view employees by department, and calculate and display employee salary information.
- **Budget Management:** Allows users to enter departmental budgets and expenditure, calculate remaining budgets, display budget information and identify departments that have exceed their allocated budgets.
- **Supplier Management:** Allows users to add, display and search registered suppliers, display field lengths and supplier description generation.
- **Asset Management:** Provides a basic asset register that allows users to add assets, display registered assets and search for assets.
- **Reports:** Provides separate reports on employee financial information, budgets, suppliers and assets.
- **Input Validation:** Validates menu choices and module inputs to prevent invalid data from being entered.

5. Program design

The Municipal Financial Management System was developed in ANSI C (C99) using a modular design. The system is divided into separate source (.c) and header (.h) files, with each module responsible for a specific area. The main.c file starts the program by calling the main menu, while menu.c and menu.h manage navigation between modules. The functional modules are:

- employees.c / employees.h - manages employee information and salary calculations.
- budget.c / budget.h - manages departmental budgets and expenditure.
- suppliers.c / suppliers.h - manages supplier information and supplier-related operations.
- assets.c and assets.h - manages the municipal assets register, including adding, displaying and searching assets.
- reports.c and reports.h – generates reports on employees, budgets, suppliers and assets.

Functions are used to divide tasks into smaller operations and reduce repetition, while parameters pass required data between functions. Arrays store multiple records, and strings handle text such as names and descriptions. Decision-making structures and loops are used for menu selection,

searching, calculations and repeated user interaction. All modules are integrated through the central menu, allowing users to access the different management areas within one program.

6. Challenges encountered

- **Integration of separate modules:** Combining the employee, budget, supplier, asset and reports modules into one application required adjustments to function declarations, variable names and module interfaces.
- **Duplicate and conflicting definitions:** The budget and reports modules contained their own main() functions, while several modules contained duplicate variables or function definitions. These conflicts had to be identified and resolved before integration.
- **Data sharing between modules:** The reports module needs access to data stored in the employee, supplier, budget and asset modules. This requires careful use of parameters and external declarations to ensure that the correct data types, arrays and variables are referenced.
- **GitHub and executable file management:** During version control, the executable file (mfmsProjectA.exe) was initially excluded by the .gitignore file. The executable file had to be deliberately added and committed.

7. Solutions implemented

The challenges encountered during the system's development were addressed through the following solutions:

- **Module integration**: The separate modules were integrated through main.c and menu.c, allowing the system to operate as one application while maintaining separate module functionality.
- **Program entry-point conflicts**: The main functions that were originally present in the budget and reports modules were replaced with module-specific menu functions. This allowed the system to use a single main() function while still providing access to individual module menus.
- **Variable and function consistency**: Conflicting variable declarations, data types and function parameters were reviewed and aligned across the source and header files. This resolved compilation and linking errors encountered during integration.

- **Data Sharing between modules**: Data required by the Reports module was passed through function parameters and appropriate external declarations were used where necessary, allowing the Reports module to access relevant employee, budget, supplier and asset information without duplicating the data.
- **GitHub integration**: Git and Github were used to track and synchronise changes during development. The executable file was explicitly added to the repository after .exe files were excluded by the .gitignore configuration.

These solutions resulted in a successfully compiled and integrated system. The completed application was tested by navigating through the Employee, Budget, Supplier, Asset and Reports menus, and exiting the system successfully.

8. Conclusion

The Municipal Financial Management System successfully integrates employee salary management, budget management, supplier management, asset management and reporting within a single menu-driven C application. The system was developed using a modular approach, allowing each functional area to be implemented separately while remaining integrated through the central menu.

The project demonstrated the use of functions, arrays, strings, decision-making structures, loops, parameters and header files in developing a structured C application. The challenges encountered during module integration, data sharing, compilation and version control were addressed to produce a successfully compiled and functional system. The completed application was tested by navigating through the different modules and confirming that the system could operate as an integrated application.